

RABIN KARP ALGORITHM FOR PATTERN SEARCHING

In the Naive String Matching algorithm, we check whether every substring of the text of the pattern's size is equal to the pattern or not one by one. Like the Naive Algorithm, the Rabin-Karp algorithm also check every substring. But unlike the Naive algorithm, the **Rabin Karp algorithm** matches the **hash value** of the pattern with the hash value of the current substring of text, and if the hash values match then only it starts matching individual characters. The worst case time complexity of robin Karp algorithm is $O(n-m+1)m$.

Example1:-

T = a c a a b a a a b
T = 1 3 1 1 2 1 1 1 2


p = 1 1 2
P = a a b

Assume

a = 1
b = 2
c = 3
d = 4

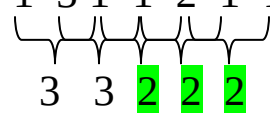
// Take addition of alphabets as hash value :- $1+1+2=4$

VH = Valid

SH = Spurious Hit

131= 5, 311=5, 112=4, 121=4, 211=4
SH SH VH VH VH

Example2:-

T = a c a a b a a
T = 1 3 1 1 2 1 1


P = 1 1 2
P = a a b

Assume

a = 1
b = 2
c = 3
d = 4

// Take multiplication as hash value

$1 * 1 * 2 = 2$

VH = Valid

SH = Spurious Hit

131= 3, 311=3, 112=2, 121=2, 211=2
SH SH VH VH VH

Rabin Karp suggest the hash function or hashing value, by random prime number q and calculate $p[1 \dots m] \bmod q$

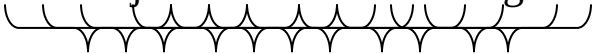
Rabin Karp Algorithm	Time Complexity
function RabinKarp($T[1..n]$, $P[1..m]$)	
hpattern := hash(pattern[1..m]);	
for i from 1 to $n-m+1$	$(n-m+1)m$
hs := hash($s[i..i+m-1]$)	
if hs = hpattern	
if $s[i..i+m-1] = \text{pattern}[1..m]$	Takes constant time but impact is $O(n)$
return i	
else	
return not found	

Example :-

Book Example :-

T= 1 2 4 9 0 2 2 1 5 1 5 2 6 7 3

T= b c e j a c c b f b f c g h d



P = f b f c g

p = 5 1 5 2 6 ;

Let a= 0; b=1; c=2; d=3; e=4; f=5;
g=6; h=7; i=8; j=9; k=10.

q = 17 // Any random prime no.

so, P mod q is = 51526 mod 17 = 16

12490 mod 17 = 12 (Spurious Hit)

24902 mod 17 = 14 (Spurious Hit)

49022 mod 17 = 11 (Spurious Hit)

90221 mod 17 = 2 (Spurious Hit)

02215 mod 17 = 5 (Spurious Hit)

22151 mod 17 = 0 (Spurious Hit)

21515 mod 17 = 10 (Spurious Hit)

15152 mod 17 = 5 (Spurious Hit)

51526 mod 17 = 16 (Valid hit)

15267 mod 17 = 1 (Spurious Hit)

52673 mod 17 = 7 (Spurious Hit)

Conclusion :-

Spurious Hit = 10

Valid Hit = 01